

ShardRoute – Enterprise Database Cluster Router

Documentation

1. Cover Page

Project Title:

ShardRoute – Enterprise Database Cluster Router

Submitted By:

Name: Ved Patil

Course: B.Tech Computer Science Engineering

Semester: II

Subject: DSA-I Using C++

Guide:

Dr. Aarti Shailendrasingh Pardeshi

2. Introduction & Problem Statement

Modern distributed databases store millions of records across multiple servers. Efficient routing of data, transaction management, replication, and load balancing are necessary for maintaining high performance and reliability.

The aim of this project is to simulate an enterprise database cluster using C++ and demonstrate how various Data Structures and Algorithms solve real-world database routing problems efficiently.

3. Objectives

- Design a distributed database routing simulator.
- Demonstrate the practical use of Data Structures and Algorithms.
- Implement fast shard routing using hash tables and tree indexing.

- Simulate transaction rollback using Stack and Linked List.
 - Process replication tasks using Priority Queue.
 - Find shortest network paths using Dijkstra's Algorithm.
 - Perform automatic storage balancing across servers.
-

4. System Architecture

The project consists of multiple database servers connected through a network. Data is divided into shards and stored on different servers. The Shard Router receives user requests, determines the correct server, processes transactions, schedules replication tasks, and balances storage.

Architecture Flow:

User Query

↓

Shard Router

↓

Database Servers (Server 0, Server 1, Server 2, Server 3)

5. Data Structures & Algorithms Used

Data Structure / Algorithm	Purpose
unordered_map	Fast shard and row lookup
map	Sorted shard range indexing
Stack	Transaction management
Singly Linked List	Undo log during rollback
Priority Queue (Min Heap)	Replication scheduling
Priority Queue (Max Heap)	Write frequency ranking
Graph (Adjacency List)	Network topology
Dijkstra Algorithm	Minimum latency routing
Vector	Store graph connections

6. Project Implementation

The project demonstrates the following modules:

- Feature 1 – Shard-to-Server Mapping

Routes user keys to the correct database server.

- Feature 2 – Transaction Rollback

Uses Stack and Linked List to undo failed transactions.

- Feature 3 – Replication Queue

Processes replication requests based on timestamps using a Min Heap.

- Feature 4 – Row Index Lookup

Uses Hash Tables for constant-time row searching.

- Feature 5 – Write Frequency Ranking

Ranks the busiest shards using a Max Heap.

- Feature 6 – Network Topology

Represents communication links between servers using Graphs.

- Feature 7 – Shortest Path Routing

Uses Dijkstra's Algorithm to calculate the minimum latency path.

- Feature 8 – Cluster Storage Balancing

Balances storage utilization across database servers.

7. Time & Space Complexity

Operation	Time Complexity	Space Complexity
Shard Lookup	$O(\log n)$	$O(S)$

Row Lookup	$O(1)$	$O(R)$
Transaction Stack	$O(1)$	$O(T)$
Linked List Insert	$O(1)$	$O(T)$
Replication Queue	$O(\log n)$	$O(Q)$
Write Frequency Heap	$O(\log n)$	$O(S)$
Dijkstra Algorithm	$O((V+E) \log V)$	$O(V+E)$
Storage Balancing	$O(S \log V)$	$O(V)$

8. Sample Output & Screenshots

Include screenshots of:

- Program execution
- Shard Mapping
- Transaction Rollback
- Replication Queue
- Row Lookup
- Write Frequency Ranking
- Network Topology
- Shortest Path Routing
- Cluster Storage Balancing

Also include sample console outputs generated by the program.

9. Results & Observations

The project successfully demonstrates the use of multiple Data Structures and Algorithms in a real-world distributed database environment.

Hash tables provide fast data retrieval, tree indexing enables efficient routing, stacks and linked lists simplify rollback operations, priority queues schedule replication tasks efficiently, and Dijkstra's Algorithm finds optimal communication paths. Storage balancing improves overall server utilization.

10. Conclusion & GitHub Repository

The ShardRoute project successfully simulates an enterprise database cluster using C++. It demonstrates practical applications of Stack, Queue, Linked List, Hash Table, Tree, Heap, Graph, and Dijkstra's Algorithm while maintaining efficient time and space complexity.

The project fulfills the objectives of the DSA-I course by applying Data Structures and Algorithms to solve a realistic problem.

GitHub Repository:

(<https://github.com/vedpatil1208-art/shard-router-dsa/tree/main>)